

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80
81     } else {
82     //it scrolled out of view
83     t.appeared = false;
84     }
85 };
86
87 //create a modified fn with some additional logic
88 var modifiedFn = function() {
89
90     //mark the element as visible
91     t.appeared = true;
92
93     //is this supposed to happen only once?
94     if (settings.one) {
95
96         //remove the check
97         w.unbind('scroll', check);
98         var i = $.inArray(check, $.fn.appear.checks);
99         if (i >= 0) $.fn.appear.checks.splice(i, 1);
100     }
101
102     //trigger the original fn
103     fn.apply(this, arguments);
104
105     //bind the modified fn to the element
106     //if (settings.one) t.one('appear', settings.data, modifiedFn);
107     //else t.on('appear', settings.data, modifiedFn);
108 };

```



MATERIA:

Programación Orientada a Objetos

Unidad 4 Persistencia, Archivos y Concurrencia Moderna

Docente:

Grupo:

Unidad #4 - Persistencia, Archivos y Concurrencia Moderna.

5. Introducción a la Programación Reactiva y Spring Cloud

5.1 El Problema: Las Limitaciones del Modelo Clásico (Blocking)

5.2 La Solución: El Paradigma Reactivo

5.3 Programación Reactiva en Java: Spring WebFlux y Project Reactor

5.4 El Siguiendo Nivel: De Monolito a Microservicios

5.5 Orquestando Microservicios con Spring Cloud

¿Qué es la Programación Reactiva?

¿Qué es la Programación Reactiva?**

- Paradigma basado en **flujos de datos**.
- Cambios se **propagan automáticamente**.
- Operaciones **asíncronas y no bloqueantes**.
- Maneja muchas solicitudes usando **menos recursos**.
- Ideal para sistemas de **alto rendimiento y tiempo real**.



El Problema: Las Limitaciones del Modelo Clásico (Blocking)

¿Qué es **Blocking**? El hilo se mantiene esperando mientras se completa una operación I/O

● Problemas Principales:

1. **Bloqueo de Hilos**
 - Un hilo = Una petición
 - Hilo BLOQUEADO esperando respuestas (DB, APIs, archivos)
 - Recursos desperdiciados durante la espera
2. **Escalabilidad Limitada**
 - Número finito de hilos disponibles
 - Bajo alta carga: hilos agotados = peticiones rechazadas
3. **Rendimiento Deficiente**
 - Context switching costoso
 - Latencias acumulativas

Ejemplo Real: Si la DB tarda 2 segundos, el hilo está bloqueado 2 segundos completos

Problema del Modelo Clásico (Blocking)

Hilos Bloqueados esperando I/O



Señales de Problemas - Diagnóstico

⚠ Indicadores Críticos de Blocking:

Síntoma	Significado	Solución
CPU baja (< 30%) pero sistema lento	Hilos esperando I/O	Reactivo
Threads en WAITING/TIMED_WAITING	Bloqueos frecuentes	Non-blocking
Timeouts frecuentes bajo carga	Pool agotado	Más threads o Reactivo
OutOfMemory con muchos threads	Demasiados hilos	Reactivo

Cuándo es Problemático:

- **X** Alta concurrencia (miles de usuarios simultáneos)
- **X** Operaciones I/O lentas (APIs externas, bases de datos)
- **X** Microservicios con muchas llamadas en cascada
- **X** Aplicaciones de streaming/tiempo real

La Solución - Paradigma Reactivo

La Programación Reactiva es un paradigma basado en flujos de datos asíncronos y propagación de cambios, donde las operaciones NO bloquean hilos.

Fórmula:

```
Programación Reactiva =  
Asíncrono + Non-Blocking + Eventos + Streams
```

Modelo Mental:

- Los datos fluyen como un río (stream)
- Reaccionas a eventos cuando ocurren
- No esperas activamente (no blocking)
- Propagación automática de cambios

Beneficios Clave:

-  Pocos threads (4-8), alta concurrencia (10,000+ requests)
-  Hilos liberados durante esperas I/O
-  Mejor uso de recursos (CPU y memoria)
-  Mayor throughput y menor latencia

Principios Reactivos Fundamentales

1. Asíncrono (Asynchronous)

```
java
// ✗ SÍNCRONO - Espera aquí
String result = database.query();
process(result);

// ✔ ASÍNCRONO - Continúa inmediatamente
database.query()
    .subscribe(result -> process(result));
```

2. Non-Blocking I/O

- Los hilos NO se bloquean esperando I/O
- Se liberan para otras tareas

3. Event-Driven (Basado en Eventos)

- `onNext(data)` → Nuevo dato disponible
- `onError(error)` → Error ocurrido
- `onComplete()` → Stream terminado

4. Backpressure

- El consumidor controla la velocidad del productor
- Evita saturación de memoria

Spring WebFlux - Tipos Reactivos

Mono<T> - 0 o 1 elemento

```
java
// Representa un único valor o vacío
Mono<User> user = userRepository.findById(1L);

Mono<String> empty = Mono.empty(); // Sin valor
Mono<String> single = Mono.just("Hello"); // Un valor
```

Flux<T> - 0 a N elementos

```
java
// Representa un stream de múltiples valores
Flux<Product> products = productRepository.findAll();

Flux<Integer> range = Flux.range(1, 100); // 1 a 100
Flux<String> list = Flux.just("A", "B", "C");
```

¿Cuándo usar cada uno?

- **Mono**: GET by ID, operaciones únicas, respuestas HTTP
- **Flux**: Listas, streams, eventos en tiempo real

REST API Reactivo - Ejemplo

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @GetMapping("/{id}")
    public Mono<Product> getProduct(@PathVariable Long id) {
        return productService.findById(id)
            .switchIfEmpty(Mono.error(
                new NotFoundException("Product not found")))
            .timeout(Duration.ofSeconds(5));
    }

    @GetMapping
    public Flux<Product> getAllProducts() {
        return productService.findAll()
            .take(100); // Limitar a 100
    }

    @PostMapping
    public Mono<Product> createProduct(
        @RequestBody Product product) {
        return productService.save(product);
    }
}
```

Características:

- Retorna **Mono<T>** o **Flux<T>**
- No bloquea threads
- Composable con operadores

Este controlador expone **3 endpoints reactivos**:

- Obtener un producto por ID (Mono)
- Obtener lista de productos (Flux)
- Crear producto (Mono)

Todo usando WebFlux: **asíncrono, no bloqueante y reactivo**.

Comparativa MVC vs WebFlux

Aspecto	Spring MVC	Spring WebFlux
Modelo	Blocking, síncrono	Non-blocking, asíncrono
API	Servlet API	Reactive Streams
Servidor	Tomcat, Jetty	Netty (default), Undertow
Return Type	<code>User</code> , <code>List<User></code>	<code>Mono<User></code> , <code>Flux<User></code>
Database	JDBC (blocking)	R2DBC (reactive)
HTTP Client	RestTemplate	WebClient
Threading	Thread-per-request	Event Loop (pocos threads)
Throughput	~500 req/s	~15,000 req/s
Aprendizaje	Fácil	Complejo

Conclusión: WebFlux para alta concurrencia, MVC para simplicidad

Cuándo Usar Reactivo

✓ CASOS IDEALES para Reactivo:

1. Alta Concurrencia

- Miles de usuarios simultáneos
- Aplicaciones con recursos limitados

2. Operaciones I/O Intensivas

- Múltiples llamadas a APIs externas
- Consultas de bases de datos lentas

3. Microservicios con Dependencias

- Servicios que llaman a otros servicios
- Cadenas de llamadas complejas

4. Streaming de Datos

- Eventos en tiempo real
- WebSockets, Server-Sent Events

✗ NO USAR Reactivo cuando:

- CRUD simple con baja carga
- Operaciones CPU-intensive (cálculos pesados)
- Equipo sin experiencia reactiva
- Bases de datos sin soporte reactivo (solo JDBC)
- Aplicaciones legacy (migración costosa)

Arquitectura Monolítica

¿Qué es un Monolito? Una aplicación donde todos los componentes (UI, lógica de negocio, acceso a datos) están integrados en una sola unidad ejecutable.

Características:

- **Deployment:** JAR/WAR único
- **Proceso:** Una JVM
- **Base de datos:** Compartida entre todos los módulos
- **Escalamiento:** Replicar TODO (ineficiente)

✓ Ventajas:

- Simplicidad inicial (fácil de desarrollar)
- Debugging directo (un solo proceso)
- Transacciones ACID simples
- Rendimiento interno excelente (sin latencia de red)

✗ Desventajas:

- Escalamiento ineficiente (todo o nada)
- Despliegues largos y riesgosos
- Código difícil de mantener conforme crece
- Tecnologías atrapadas en decisiones tempranas
- Un error puede tumbar todo el sistema

Arquitectura de Microservicios

Los microservicios descomponen la aplicación en servicios pequeños e independientes, cada uno ejecutándose en su propio proceso.

Principios Fundamentales:

Responsabilidad Única

- Cada servicio = Una capacidad de negocio
- Ejemplo: User Service, Order Service, Payment Service

Autonomía Completa

- Independiente en desarrollo, despliegue y escalamiento
- Cada servicio puede evolucionar sin afectar otros

Base de Datos por Servicio

- Cada servicio gestiona su propia BD
- Datos compartidos vía APIs, no acceso directo

Diseñado para Fallar

- Asume que las llamadas remotas pueden fallar
- Implementa timeouts, reintentos, circuit breakers

Características Técnicas:

- Comunicación: HTTP/REST, gRPC, mensajería
- Despliegue: Contenedores (Docker), orquestación (Kubernetes)
- Tecnología: Polyglot posible (Java, Node.js, Go, Python)

Comparativa Monolito vs Microservicios

Dimensión	Monolito	Microservicios
Estructura	Una aplicación	Múltiples servicios independientes
Datos	Base de datos compartida	Base por servicio
Deployment	Todo junto (big bang)	Independiente por servicio
Escalamiento	Vertical (toda la app)	Horizontal (por servicio)
Complejidad	Simple inicialmente	Compleja desde inicio
Resiliencia	Fallo global (todo cae)	Fallo aislado
Tecnología	Stack único	Polyglot posible
Team Structure	Equipo único	Equipos autónomos
Desarrollo	Rápido al inicio	Lento al inicio, rápido después

Cuándo Migrar:

- Monolito demasiado grande para un equipo
- Necesitas escalar partes específicas
- Diferentes partes requieren tecnologías distintas
- Quieres releases independientes

Comunicación entre Servicios

Comunicación Síncrona (REST/HTTP):

```
java
// Order Service llamando a Product Service
@Service
public class OrderService {
    private final WebClient webClient;

    public Mono<Product> getProductInfo(Long productId) {
        return webClient.get()
            .uri("http://product-service/api/products/{id}",
                productId)
            .retrieve()
            .bodyToMono(Product.class)
            .timeout(Duration.ofSeconds(5))
            .onErrorResume(e -> {
                log.error("Product service unavailable", e);
                return Mono.just(Product.getDefault());
            });
    }
}
```

Comunicación Asíncrona (Eventos):

```
java
// Publicar evento
@Service
public class OrderService {
    public void createOrder(Order order) {
        orderRepository.save(order);

        // Publicar evento asíncrono
        OrderCreatedEvent event = new OrderCreatedEvent(
            order.getId(), order.getUserId(), order.getTotal()
        );
        rabbitTemplate.convertAndSend("orders.exchange",
            "order.created", event);
    }
}
```

Cuándo usar cada tipo:

- **Síncrona:** Respuesta inmediata (consultas, validaciones)
- **Asíncrona:** Eventos de negocio, operaciones largas, desacoplamiento

Spring Cloud - Componentes Principales

Ecosistema Completo:

Componente	Propósito	Problema que Resuelve
Eureka	Service Discovery	¿Dónde están mis servicios?
Config Server	Configuración centralizada	¿Cómo gestionar configs?
Gateway	API Gateway	¿Cómo unificar punto de entrada?
Resilience4j	Circuit Breaker	¿Cómo manejar fallos?
Sleuth	Distributed Tracing	¿Cómo rastrear requests?
Stream	Event-Driven	¿Cómo comunicar asincrónamente?
OpenFeign	HTTP Client	¿Cómo simplificar llamadas REST?
LoadBalancer	Client-side LB	¿Cómo distribuir carga?

Beneficio: Simplifica la construcción de sistemas distribuidos

Eureka Server - Service Discovery

¿Por qué lo necesitamos? En microservicios, los servicios cambian de ubicación constantemente. Hardcodear IPs/puertos es imposible.

```
java
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

```
properties
# application.properties
server.port=8761
spring.application.name=eureka-server

# No registrarse a sí mismo
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

Dashboard: <http://localhost:8761>

Función: Registro centralizado donde los servicios se registran y descubren

Registrar Microservicio en Eureka

Cliente Eureka:

```
java
@SpringBootApplication
@EnableDiscoveryClient // ← Habilita cliente
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}
```

```
properties

# application.properties
spring.application.name=user-service
server.port=8081

# URL del Eureka Server
eureka.client.service-url.defaultZone=
    http://localhost:8761/eureka/

# Configuración de heartbeat
eureka.instance.lease-renewal-interval-in-seconds=30
eureka.instance.lease-expiration-duration-in-seconds=90
```

Flujo:

1. Servicio se registra al iniciar
2. Envía heartbeats cada 30s
3. Otros servicios consultan Eureka para encontrarlo

API Gateway con Spring Cloud Gateway

¿Por qué un Gateway? Sin gateway, los clientes deben conocer múltiples microservicios. Con gateway, un único punto de entrada.

Responsabilidades del Gateway:

-  **Routing:** Dirigir requests al servicio apropiado
-  **Autenticación/Autorización:** Validar tokens, permisos (una sola vez)
-  **Rate Limiting:** Proteger contra abuso
-  **Request/Response Transformation:** Agregar headers, modificar formato
-  **Caching:** Respuestas frecuentes
-  **Logging/Monitoring:** Centralizar observabilidad

Configuración Básica:

```
yaml
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service # lb = load balanced
          predicates:
            - Path=/api/users/**
          filters:
            - RewritePath=/api/users/(?<segment>.*), /${segment}

        - id: product-service
          uri: lb://product-service
          predicates:
            - Path=/api/products/**
```

Beneficios:

- Simplifica clientes (un solo endpoint)
- Centraliza cross-cutting concerns
- Oculta estructura interna

Patrones de Resiliencia

1. Circuit Breaker (Resilience4j)

```
@Service
public class OrderService {

    @CircuitBreaker(name = "productService",
        fallbackMethod = "getDefaultProduct")
    public Mono<Product> getProduct(Long id) {
        return webClient.get()
            .uri("http://product-service/api/products/{id}", id)
            .retrieve()
            .bodyToMono(Product.class);
    }

    // Fallback cuando el circuit está abierto
    public Mono<Product> getDefaultProduct(Long id, Exception e) {
        return Mono.just(new Product(id, "Unavailable", 0.0));
    }
}
```

Estados del Circuit Breaker:

- **CLOSED:** Normal, permite todas las llamadas
- **OPEN:** Rechaza llamadas, retorna fallback
- **HALF-OPEN:** Prueba recuperación

2. Retry Pattern

```
java

@Retry(name = "productService", fallbackMethod = "getDefaultProduct")
public Mono<Product> getProduct(Long id) {
    return webClient.get()...
}
```

3. Timeout Pattern

```
java

public Mono<Product> getProduct(Long id) {
    return webClient.get()...
        .timeout(Duration.ofSeconds(5));
}
```

Event-Driven con Spring Cloud Stream

¿Por qué Eventos? La comunicación síncrona (REST) crea acoplamiento temporal. Para eventos de negocio, necesitamos mensajería.

Spring Cloud Stream - Functional Programming:

```
java
@Configuration
public class OrderEvents {

    // Publicar eventos (Producer)
    @Bean
    public Supplier<Flux<OrderCreatedEvent>> orderPublisher() {
        return () -> Flux.interval(Duration.ofSeconds(1))
            .map(i -> new OrderCreatedEvent(i, "user-" + i));
    }

    // Consumir eventos (Consumer)
    @Bean
    public Consumer<OrderCreatedEvent> orderConsumer() {
        return event -> {
            log.info("Processing order: {}", event.getOrderID());
            inventoryService.reserve(event.getOrderID());
        };
    }

    // Transformar eventos (Processor)
    @Bean
    public Function<OrderCreatedEvent, OrderProcessedEvent>
        orderProcessor() {
        return orderCreated -> {
            // Procesar lógica
            return new OrderProcessedEvent(
                orderCreated.getOrderID()
            );
        };
    }
}
```

Configuración:

```
spring:
  cloud:
    stream:
      bindings:
        orderPublisher-out-0:
          destination: orders
        orderConsumer-in-0:
          destination: orders
          group: inventory-service
```

Monitoreo con Spring Boot Actuator

Actuator Endpoints para Observabilidad:

```
yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: always
health:
  circuitbreakers:
    enabled: true
  ratelimiters:
    enabled: true
```

Endpoints Útiles:

Endpoint	Información
<code>/actuator/health</code>	Estado del servicio y dependencias
<code>/actuator/metrics</code>	Métricas de rendimiento (JVM, HTTP, etc)
<code>/actuator/prometheus</code>	Formato Prometheus para scraping
<code>/actuator/info</code>	Información del servicio (versión, etc)
<code>/actuator/env</code>	Variables de entorno

Health Check Personalizado:

```
@Component
public class DatabaseHealthIndicator
    implements HealthIndicator {

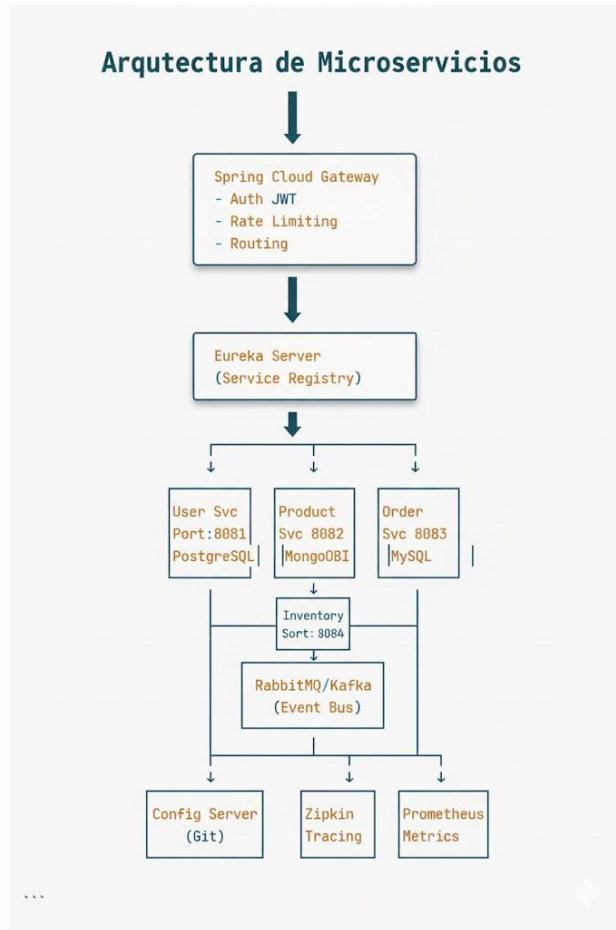
    @Override
    public Health health() {
        if (isDatabaseUp()) {
            return Health.up()
                .withDetail("database", "PostgreSQL")
                .withDetail("version", "14.5")
                .build();
        }
        return Health.down()
            .withDetail("error", "Cannot connect")
            .build();
    }
}
```

Este código crea un **HealthIndicator personalizado** para Spring Boot Actuator.

- **DatabaseHealthIndicator** verifica si la base de datos está funcionando (**isDatabaseUp()**).
- Si la base de datos responde:
 - Devuelve **Health.up()** con detalles como:
 - Motor: PostgreSQL
 - Versión: 14.5
- Si la base de datos *no* responde:
 - Devuelve **Health.down()** indicando que no hay conexión.

Este componente permite que Actuator reporte el **estado de la base de datos** (UP o DOWN) en el endpoint **/actuator/health**.

Arquitectura Completa - Diagrama de Flujo



Microservicios Modernos: Lo Esencial

- **API Gateway:** Es la **puerta de entrada única** para todos los clientes (Web/Mobile). Se encarga de la **Autenticación** y el **Enrutamiento** de peticiones.
- **Eureka Server:** Actúa como el **Registro de Servicios**. Permite que los microservicios se **descubran** y se comuniquen entre sí sin depender de IPs fijas.
- **Servicios Políglotas:** La lógica de negocio está separada en servicios independientes. Cada uno puede usar la **mejor tecnología** (Java, Node.js, Go) y su **propia base de datos** (MongoDB, MySQL) para su tarea.
- **Event Bus:** Utiliza plataformas como Kafka o RabbitMQ para la comunicación **asíncrona y no bloqueante**. Esto es clave para la **resiliencia** y la **escalabilidad**.
- **Infraestructura de Observabilidad:** Incluye herramientas vitales como **Zipkin** (para rastrear transacciones a través de múltiples servicios) y **Prometheus** (para monitorear métricas y rendimiento).

Estrategias de Migración - Strangler Fig

Big Bang: Reescribir todo (muy riesgoso)

Strangler Fig: Migración gradual

Pasos del Strangler Fig:

1. Identificar bounded context para extraer
2. Crear nuevo microservicio
3. Routing configurable (porcentaje de tráfico al nuevo servicio)
4. Aumentar tráfico gradualmente (10% → 50% → 100%)
5. Remover código del monolito una vez estable

Ejemplo:

Semana 1: 10% tráfico → Microservicio (90% → Monolito) Semana 2: 50% tráfico → Microservicio (50% → Monolito) Semana 3: 100% tráfico → Microservicio (0% → Monolito)

Beneficios:

- Validar cada migración antes de continuar
- Rollback fácil si hay problemas
- Bajo riesgo

Gestión de Datos Distribuidos - Saga Pattern

El Desafío:

En monolitos: Transacciones ACID En microservicios: Datos distribuidos, ¿cómo mantener consistencia?

Solución: Saga Pattern

Coordinar transacciones distribuidas como secuencia de transacciones locales + compensaciones

Ejemplo: Crear Orden

1. Order Service: Crear orden ✓
2. Payment Service: Cargar pago ✓
3. Inventory Service: Reservar stock ✗ (FALLO)

→ Compensar:

- Payment Service: Reembolsar
- Order Service: Cancelar orden

Dos implementaciones:

Choreography (Descentralizado):

- Cada servicio escucha eventos y decide su acción
- Descentralizado pero difícil de rastrear

Orchestration (Centralizado):

```
@Service
public class OrderSaga {
    public Mono<Void> executeCreateOrderSaga(Order order) {
        return createOrder(order)
            .flatMap(this::processPayment)
            .flatMap(this::reserveInventory)
            .flatMap(this::scheduleShipping)
            .onErrorResume(error ->
                compensateOrder(order, error)
            );
    }
}
```

Observabilidad Distribuida - Distributed Tracing

El Problema: En monolitos: Stack trace completo En microservicios: Request atraviesa 10+ servicios, ¿cómo rastrear?

Solución: Distributed Tracing (Zipkin/Jaeger)

Conceptos:

- **Trace ID:** Identificador único de request end-to-end
- **Span ID:** Identificador de operación en un servicio
- **Parent Span ID:** Para construir jerarquía

Implementación con Spring Cloud Sleuth:

```
java
// Dependencia
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-sleuth-zipkin</artifactId>
</dependency>
```

```
properties

spring.zipkin.base-url=http://localhost:9411
spring.sleuth.sampler.probability=1.0 # 100% traces
...

**Logs automáticos:**
...

[app-name,traceId,spanId] INFO - Processing request
[user-service,abc123,def456] INFO - Finding user 1
[order-service,abc123,ghi789] INFO - Creating order
```

Visualización: Zipkin UI muestra el flujo completo con latencias

WebClient - Cliente HTTP Reactivo

Reemplazo de RestTemplate:

```
@Configuration
public class WebClientConfig {

    @Bean
    public WebClient webClient(WebClient.Builder builder) {
        return builder
            .baseUrl("https://api.example.com")
            .defaultHeader("Content-Type", "application/json")
            .filter(logRequest())
            .filter(retryFilter())
            .build();
    }
}

@Service
public class ProductService {

    private final WebClient webClient;

    // GET Request
    public Mono<Product> getProduct(Long id) {
        return webClient.get()
            .uri("/products/{id}", id)
            .retrieve()
            .bodyToMono(Product.class)
            .timeout(Duration.ofSeconds(5))
            .onErrorResume(e -> {
                log.error("Product service unavailable", e);
                return Mono.just(Product.getDefault());
            });
    }

    // POST Request
    public Mono<Product> createProduct(Product product) {
        return webClient.post()
            .uri("/products")
            .bodyValue(product)
            .retrieve()
            .bodyToMono(Product.class);
    }

    // GET List
    public Flux<Product> getAllProducts() {
        return webClient.get()
            .uri("/products")
            .retrieve()
            .bodyToFlux(Product.class);
    }
}
```

Ventajas sobre RestTemplate:

- Non-blocking
- Composable con operadores reactivos
- Streaming support
- Mejor manejo de errores

R2DBC - Base de Datos Reactiva

JDBC vs R2DBC:

JDBC	R2DBC
Blocking	Non-blocking
Thread-per-connection	Alta concurrencia
Established (1997)	Nuevo (2018)

Configuración:

```
spring:
  r2dbc:
    url: r2dbc:postgresql://localhost:5432/userdb
    username: admin
    password: secret
```

Repository Reactivo:

```
java
public interface UserRepository
    extends ReactiveCrudRepository<User, Long> {

    Flux<User> findByName(String name);
    Mono<User> findByEmail(String email);
}

java
@Service
public class UserService {

    private final UserRepository userRepository;

    public Mono<User> findById(Long id) {
        return userRepository.findById(id)
            .switchIfEmpty(Mono.error(
                new NotFoundException("User not found")
            ));
    }

    public Flux<User> findAll() {
        return userRepository.findAll();
    }

    public Mono<User> save(User user) {
        return userRepository.save(user);
    }
}
```

Bases de datos soportadas:

- PostgreSQL (r2dbc-postgresql)
- MySQL (r2dbc-mysql)
- H2 (r2dbc-h2)
- SQL Server (r2dbc-mssql)

Operadores Reactivos Comunes

Transformación:

```
java
// map - Transformar sincronamente
Flux.just(1, 2, 3)
    .map(x -> x * 2) // 2, 4, 6

// flatMap - Transformar asincrónamente
Flux.just(1, 2, 3)
    .flatMap(x -> service1.call(x)
        .flatMap(y -> service2.call(y)))

// filter - Filtrar elementos
Flux.range(1, 10)
    .filter(x -> x > 5) // 6, 7, 8, 9, 10
```

Manejo de Errores:

```
java
// onErrorReturn - Valor por defecto
mono.onErrorReturn(defaultValue)

// onErrorResume - Fuente alternativa
mono.onErrorResume(e -> fallbackMono)

// retry - Reintentar
mono.retry(3)

// timeout - Timeout máximo
mono.timeout(Duration.ofSeconds(5))
```

Combinación:

```
java
// zip - Combinar resultados
Mono.zip(mono1, mono2)
    .map(tuple -> tuple.getT1() + tuple.getT2())

// merge - Intercalar (paralelo)
Flux.merge(flux1, flux2)

// concat - Secuencial
Flux.concat(flux1, flux2)
```

Limitación:

```
java
.take(10)        // Primeros 10
.skip(5)         // Saltar primeros 5
.takeLast(3)     // Últimos 3
.distinct()      // Elementos únicos
```


Schedulers - Control de Threading

Scheduler	Características	Uso
<code>immediate()</code>	Thread actual, sin cambio	Operaciones rápidas
<code>single()</code>	Un solo thread compartido	Tareas secuenciales
<code>parallel()</code>	Pool fijo (CPU cores)	CPU-intensive
<code>boundedElastic()</code>	Pool elástico con límite	Blocking inevitable (DB legacy, File I/O)

subscribeOn() vs publishOn():

```
java
Flux.range(1, 5)
    .map(i -> {
        log.info("map1: " + Thread.currentThread().getName());
        return i * 2;
    })
    .subscribeOn(Schedulers.boundedElastic()) // + Afecta TODO upstream
    .map(i -> {
        log.info("map2: " + Thread.currentThread().getName());
        return i + 1;
    })
    .publishOn(Schedulers.parallel()) // + Afecta solo downstream
    .map(i -> {
        log.info("map3: " + Thread.currentThread().getName());
        return i * 3;
    })
    .subscribe();
---
```

****Salida:****

map1: boundedElastic-1
map2: boundedElastic-1
map3: parallel-1

Regla:

- `subscribeOn()`: Dónde ocurre la **suscripción** (solo el primero tiene efecto)
- `publishOn()`: Dónde se **procesan señales** (pueden haber múltiples)